

OS-Homework10

计算机 2303 尤颢辰

2026 年 5 月 16 日

1 Question 1:

可能出现的问题:

1. 当原文件的磁盘空间被回收并分配给新文件后, 所有仍指向该存储区域的旧链接就变成了**悬空引用**。此时通过旧链接读取数据, 得到的是新文件的内容; 若通过旧链接写入, 则会覆盖新文件的数据。这会导致原文件所有者以为数据已被删除, 但持有旧链接的进程却能读取到新文件的内容。

2. 若新文件复用了被删除文件的路径名, 所有指向该路径的软链接将自动重新生效并指向新文件, 持有链接的进程不会察觉目标已经改变, 从而在**错误的文件上执行读写操作**。

解决方案:

最直接的解决方法是引用计数: 在 inode 中记录当前指向该文件的链接数与打开该文件的进程数之和, 只有计数归零时才真正回收磁盘空间, 从而保证只要还有任何引用存在, 文件数据就不会被释放。

针对路径复用问题, 可以为每个文件分配一个唯一的不可复用标识符 (如 generation number) 并记录在链接中, 每次访问时对比该标识符与当前 inode 是否一致, 不匹配则拒绝访问。这样即使新文件复用了相同的路径或 inode 编号, 旧链接也无法通过校验。

2 Question 2:

操作系统应同时维护两级表结构: **一张系统级的全局开放文件表, 以及每个进程独立的进程级开放文件表**。全局表记录所有当前被访问的文件的共享信息, 如 inode、磁盘位置、文件大小等; 进程级表则记录各进程私有的状态, 如当前读写偏移量 (file offset) 和访问权限, 并通过指针引用全局表中的对应条目。这样的两级设计既避免了重复存储文件的元数据, 又允许不同进程对同一文件维护各自独立的读写状态。

当同一个文件被两个不同的程序或用户同时访问时, **全局表中该文件只需保留一个条目, 但每个进程在其进程级表中应有各自独立的条目**。这是因为两个进程对同一文件的读写进度通常是相互独立的, 若共用同一个偏移量, 一个进程的读写操作将会干扰另一个进程, 导致数据错乱。

3 Question 3:

强制锁:

强制锁由操作系统内核强制执行, 任何进程在访问已加锁的文件时都会被自动阻塞或拒绝, 无需程序员在代码中主动检查锁的状态。其优点在于安全性更高, 锁的保护是全局生效的, 不存在某个进程忘记检查锁便

直接访问文件的情况。其缺点也很明显：内核必须在每次文件访问时都介入检查，带来额外的性能开销；同时灵活性较差，若某个进程持有锁后崩溃，其他进程将被无限期阻塞，容易引发死锁。

建议锁：

建议锁内核并不强制执行，进程需要在访问文件前主动检查并遵守锁的约定。其开销小、灵活性高，程序可以根据具体需求决定加锁策略，内核无需在每次 I/O 时都去检查。但缺点是安全性完全依赖所有进程自觉遵守，只要有一个进程不检查锁便直接访问文件，锁机制就不能生效。

4 Question 4:

优点：跨挂载点链接允许用户将不同卷上的文件统一组织在同一目录结构下，用户不用关心文件实际存储在哪个卷上。这在大型系统中有优势，例如我们可以将网络文件系统，或外部存储设备中的文件整合进本地的目录树。

缺点：首先其可靠性不足，若目标卷未被挂载或被卸载，链接将变为悬空引用，访问时报错，且用户难以判断问题根源。其次，硬链接跨卷在技术实现难度大，因为硬链接依赖 inode 编号，而 inode 仅在单一文件系统内有意义，不同卷之间的 inode 编号是相互独立的。因此跨挂载点的链接通常只能以软链接的形式存在。此外，文件系统的备份与迁移也会变得更加复杂，若只备份某一个卷，恢复后无法保证数据的完整性。